Scriptorium

Scripture shorts, in your own language — and a build that fails rather than ship an altered verse.

Technical writeup · Dr. Philemon Paul Daniel · *Scripture in New Frontiers* (Gloo × YouVersion), August 2026
Live studio scriptorium-gamma-wheat.vercel.app · Source github.com/phildani7/scriptorium (Apache-2.0) · Video youtu.be/VRy04QGBpC8 · MCP `/api/mcp`

1. The problem, stated precisely

YouVersion serves Scripture as **text** in over a thousand languages. The medium the world actually spends its attention on is **vertical video**. In most of those languages there is essentially none of it.

This is not a content-appetite problem. It is a production problem. A short that anyone would watch needs three scarce people — a designer, a voice, and an editor — and a volunteer church in Dhaka or Chennai has none of them. English has thousands of Scripture shorts. Bengali has almost none. The gap is not theological; it is a shortage of production capacity, and production capacity is the kind of thing software is good at manufacturing.

Scriptorium is a creator tool that supplies all three roles. Type a reference or a feeling ("anxiety at work"), paste a sermon, drop an article link, upload a PDF, or ask for a seven-day series. Pick a lens and a language. About a minute later there is a publishable 1080×1920 short — narrated in a real voice, captioned word by word, set in the correct script, with real motion design.

2. The architectural claim

> *Scripture is retrieved, never generated — and the build proves it.*

Every design decision below follows from one constraint. A generative system that puts words on a screen next to a verse is one hallucination away from publishing altered Scripture to a person who trusts it. Disclaimers do not fix this. Prompt engineering does not fix this. The only thing that fixes it is making the failure *mechanically impossible to ship*.

So the system is split down the middle. **YouVersion supplies the verse. Gloo writes the teaching around it.** The model never sees a request that would let it produce Scripture, and the build refuses to emit a file if the verse on screen differs by one character from what the API serves.

The pipeline

```
input ──▶ /api/resolve    YouVersion     reference or topic ──▶ passage (verbatim)
      ──▶ /api/extract    Gloo AI        text/PDF/article ──▶ teachings (references only)
      ──▶ /api/series     Gloo AI        theme + days ──▶ planned series (references only)
      ──▶ /api/generate   Gloo AI        passage ──▶ 3–7 teaching devices
      ──▶ review          human          every field editable EXCEPT the verse
      ──▶ /api/compose    Speechmatics   narration + word timings ──▶ ShortSpec
      ──▶ /api/preview    HyperFrames    ShortSpec + theme ──▶ composition HTML
      ──▶ /api/export     Actions –or– Sandbox ──▶ MP4 ──▶ /gallery
      ──▶ /api/mcp        MCP            the whole pipeline as 8 stateless tools
```

Every input path converges on the same rule: **models return references only**; verse text is fetched from YouVersion afterwards. A pasted sermon, an uploaded PDF, a linked article, a planned series — none of them can put a single generated word on screen as Scripture, because none of them carry verse text forward. They carry a citation, and the citation is resolved against the API.

3. The integrity gate

The gate is `src/lib/verify/verbatim.ts`. It is **dependency-free on purpose**, so a judge can audit it in one sitting. It normalises both sides and diffs them:

- **NFC normalisation.** Devanagari matras and Tamil vowel signs have multiple valid encodings; a browser text node may hand back a different one than the API did. Without NFC the gate would fire constantly on exactly the languages this project exists to serve.
- **Render artefacts stripped** — BOM, soft hyphen, bidi marks. Things a *layout engine* legitimately inserts.
- **ZWJ and ZWNJ deliberately preserved.** In Devanagari, Bengali and Tamil these control conjunct formation and are part of the meaning. Stripping them would make the diff pass more often — which is precisely why it would be the wrong thing to do.

It runs twice. Client-side it lights the "verse verified" badge in the preview. In `render/render.ts` it runs after the DOM is populated and **before a single frame is captured**, where a mismatch throws `ScriptureIntegrityError` and kills the build. A short that ships altered Scripture is worse than no short, so the failure mode is a dead build, not a warning in a log.

The gate fails closed, which is the part that matters

A gate that silently skips when its dependency is unavailable is theatre. This one refuses in every degraded case:

Condition	Behaviour
No `YVP_APP_KEY`, or no network	Build fails. There is nothing authoritative to compare against, so it will not guess.
Spec text ≠ live API response	Build fails — "SPEC TAMPERING DETECTED".
Rendered DOM ≠ live API response	Build fails — "Refusing to render altered Scripture".
Gate-marked node missing entirely	Build fails. An absent check is not a passed check.
Teaching format (verse cited, not shown)	Check *inverts*: the marked node must be **empty**, so nothing can masquerade as Scripture.

Critically, the comparison is never spec-against-spec. `render.ts` re-fetches the passage from YouVersion *independently* and diffs the DOM against that fresh response. Tampering with the spec file does not help, because the spec is not the authority.

`npm run prove:gate` performs the whole tamper-and-refuse cycle end to end. The test suite (`verbatim.test.ts`, 13 tests) includes a single dropped Devanagari matra (`मुझे` → `मुझ`, visually near-invisible at 1080×1920 and meaning-changing) and a danda swapped for a full stop.

4. How the YouVersion Platform API is used

Base `https://api.youversion.com/v1`, authenticated with an app key. It is used for four distinct jobs, not just verse lookup:

- **Retrieval** — reference or topic to passage, verbatim, with USFM identity. Book-name parsing is local (`usfm.ts`) across the language registry, so "यूहन्ना 3:16" resolves without a round trip.
- **Version licensing** — which translations this key may actually serve per language. The audit publishes real numbers rather than platform totals: **40 languages, 116 versions**. When a reference is missing from the preferred version, the resolver tries every *licensed* version before reporting it missing.
- **Provenance** — `attribution` travels with the text through compose, render and into the gallery manifest. Every short can say which publisher's translation it quotes.
- **Verification** — the second, independent fetch at render time that the gate compares against.

Without a key the app serves clearly-labelled offline sample passages. It never asks a model for verse text as a fallback — that would be the one shortcut that destroys the premise.

5. How the Gloo AI Studio API is used

OAuth2 client credentials against `platform.ai.gloo.com`, tokens cached in module scope and refreshed early rather than paying a round trip per request. Two Gloo-specific capabilities are used deliberately rather than incidentally — this is the difference between "we called an LLM" and "we used this platform because of what it is":

- `tradition` **— values alignment.** A short for a Catholic parish and one for a Pentecostal youth group want different emphases from the same verse. Most tooling flattens everyone into one homogenised voice. The user's tradition is passed through, so the teaching is aimed at the community it is actually for. This is the capability that made Gloo the right choice rather than a generic chat endpoint: it is inference built for ministry context, and ministry context is not one context.
- `auto_routing` **— model selection with reported confidence.** Gloo picks the model per request and reports its tier and confidence. Both are recorded in the run manifest, so every short in the gallery can say which model wrote its device and how confident the router was. Provenance for the generated half, mirroring the provenance carried for the retrieved half.

Gloo drives four endpoints — `/api/generate` (passage → 3–7 teaching devices through one of six lenses), `/api/extract` (sermon, PDF or article → teachings, references only), `/api/series` (theme + days → a planned arc), and the decline path, which politely refuses non-Christian source documents rather than producing something inappropriate from them.

The output schema is strict and validated: a device missing a non-empty `explanation` is rejected rather than passed downstream. Structure is enforced at the boundary, not hoped for.

6. Challenges, and how they were worked through

6.1 Headless Chrome has no system font fallback

In a browser, a missing font degrades to something readable. In the headless Chrome that captures the MP4 there is no such fallback — the text renders **blank**. A tool whose entire claim is "Scripture in your own language" cannot delegate that to the host machine. Sixteen SIL OFL families are now self-hosted, and the render bundler copies *only* the script a given short is set in: the CJK families alone are 19 MB, and a Telugu

short has no use for 124 Japanese subsets. A CDN reference would have been simpler and would have silently shipped empty frames.

6.2 Type that fits, measured rather than guessed

A verse that overflows its panel is invisible at the bottom of the frame and nobody notices until it ships. `npm run check:fit` sweeps **252 theme × style × script configurations** across seven scripts — including Hebrew, Arabic and Han — and fails the build on any type that overflows. Text is re-fitted every time the font set finishes loading rather than once, because measuring before `document.fonts.ready` measures the fallback face and produces confidently wrong metrics.

6.3 Six pages, exactly one sentence at a time

The format is five teaching sentences, one per page, then the verse. "One at a time" is easy to assert and easy to break during a transition. `npm run smoke:pages` bakes every template and asserts that **exactly one** sentence is visible at each page boundary, capturing frames to disk for inspection. It is run after any change to a template timeline.

6.4 YouTube captions cannot be scraped from the watch page

The link reader accepts YouTube URLs as a teaching source. The obvious implementation *appears* to work: `ytInitialPlayerResponse` still contains `captionTracks` with plausible `baseUrl`s. Fetch one server-side and it returns **HTTP 200 with a zero-byte body** — the URLs are bound to the browser session, and the failure looks like success at every layer that usually tells you otherwise. The fix was InnerTube (`POST /youtubei/v1/player`) with the **iOS** client, which serves json3, falling back to Android, which serves timedtext XML. The WEB client returns `playabilityStatus: UNPLAYABLE` and zero tracks. This cost the most debugging time of anything in the project, because nothing errored.

6.5 A streaming WAV lies about its own length

Speechmatics' TTS writes its WAV before it knows its own length, so the `data` chunk carries the "unknown length" placeholder `0xFFFFFFFF`. Taken at face value that is ~268,435 seconds — three days of composition for four seconds of speech. The parser now bounds the declared size by the bytes that actually follow. (A related trap: reading the byte rate at offset +12 yields the *sample rate*, which for 16-bit mono is exactly half — every duration comes out twice as long, quietly, and plausibly.)

6.6 A silent MP4 is worse than a missing one

A Mandarin short shipped fifteen seconds of silence to the gallery: `zh` is registered with a Piper voice, synthesis failed, the job carried on, and the only trace was a warning in a CI log. The finished file was indistinguishable from a narrated one. The runner now distinguishes "this language has no voice model" (legitimate — the short shows its words and says so) from "the registry promised a voice and synthesis failed" (dishonest), and **fails the job** in the second case. Same reasoning as the verbatim gate, applied to audio.

6.7 Captions must never show what ASR heard

Speechmatics returns both *when* each word was spoken and *what* it thought the word was. The second is thrown away. Captions sit directly under a verse; if ASR mishears and we render what it heard, a viewer sees altered Scripture, and no gate elsewhere would catch it because the verse element itself was never touched. So caption text is always the script we already hold, and ASR contributes nothing but a clock. That turns captioning into a forced-alignment problem — walk script and ASR together, anchor what matches,

interpolate the rest weighted by word length. A failed match costs timing precision on one word. It can never cost accuracy.

6.8 Two render backends, kept alive side by side

Rendering runs on GitHub Actions by default, with a Vercel Sandbox microVM as an alternate, selectable per request. This looks like redundancy for its own sake until you consider that a contest demo depending on one CI queue being healthy is a demo that fails at the wrong moment. Both paths produce byte-identical compositions because both consume the same `bakeComposition` output — the preview cannot flatter the export.

7. Why these choices were right

Retrieval over generation, enforced in the build. The alternative — instructing the model to quote accurately and checking samples — fails open. It works until the one time it does not, and that time ships. Making the check a build gate converts a probabilistic risk into a deterministic one.

A dependency-free gate. The most important 180 lines in the repo import nothing. That is a deliberate trade of convenience for auditability: a reviewer can read it top to bottom and satisfy themselves it does what is claimed, without trusting a supply chain.

Server-side spec resolution. Visuals, timings and theme are resolved once, server-side, so preview and export consume one artefact. The common failure in this class of tool is a preview that lies; the architecture removes the opportunity rather than testing for it.

Self-hosted everything the render touches. Fonts, music, motion runtime, backgrounds. The render runs offline from `file://`; anything fetched at capture time is a frame that might be blank.

MCP as a first-class surface. The whole pipeline is eight stateless tools. For the "creator tools" frontier this matters: the studio can live inside the editor or agent a creator already uses, rather than demanding they come to a website. The notebook accompanying this submission drives the live production deployment entirely through that endpoint, with no credentials.

8. What is verified, and what is not

Claims are cheap; the repository ships the checks that back them.

Command	What it proves
`npm test`	Integrity-gate, USFM and alignment suites (36 tests)
`npm run prove:gate`	Tampers with a verse end to end; asserts the render refuses
`npm run check:fit`	252 theme × style × script combinations; fails on overflow
`npm run smoke:pages`	Exactly one sentence visible at every page boundary
`npm run smoke:mcp`	Speaks the real MCP protocol to a deployment and calls the tools
`npm run smoke:link`	URL reader against live YouTube / article / failure cases
`npm run audit:languages`	Re-audits language coverage against the live API

Honest limitations, stated rather than hidden. Of 40 languages, **33 are complete** (licensed text, a free neural voice, and word timing *measured* from the audio); **4 are voiced** with timing estimated from word

length; **3 are captions-only** because no free voice model exists yet. The studio preview is silent for every non-English language by design — Piper runs inside the export job, not in a serverless function — and the UI says so rather than letting a creator discover it in the finished file. Rate limiting is in-memory and therefore per-instance: a speed bump in front of paid APIs, not a wall, and documented as such.

A self-audit of the pipeline (`PIPELINE-AUDIT.md` in the repository) records the defects found by reading the code adversarially, including two preview/export parity risks, ranked and unfixed rather than quietly patched over. Publishing that alongside the submission seemed more in keeping with the project's argument than not publishing it.

9. Where this goes

The gallery holds **29 rendered shorts across 8 languages**, each of which passed the gate before its first frame was captured. The interesting number is not 29; it is that the marginal cost of the thirtieth, in a language with no Scripture video at all, is about a minute and no specialist labour.

That is the whole argument. Scripture has always travelled in whatever medium the culture was actually using — scroll, codex, print, radio. It is now late to vertical video in most of the world's languages, and the reason is production capacity rather than desire. This is a machine for manufacturing that capacity, built so it cannot lie about what it is quoting.
